

Motarda Graphic Engine

PROGRAMACIÓN DE MOTORES GRÁFICO
Advanced Rendering & Visualization

Arnau Rosselló
2025/2026

David Rial Pérez, Iván Galán Sobrino



1.- Basic information	1
2.- Problem solving	8
3.- Rendering techniques documentation	17
4.- Task distribution	32
5.- Bibliography	33



1.- Basic information

1. CORE ENGINE ARCHITECTURE

The core architecture orchestrates system initialization by bootstrapping a GLFW context, configuring asynchronous logging via spdlog, and instantiating subsystems for input processing and thread-pool execution. It handles spatial presentation via a camera system with adjustable view frustum parameters alongside procedural base-plane geometry. Crucially, it exposes standard lifecycle hooks to govern frame execution, buffer swapping, and runtime debug toggles.

2. CROSS-PLATFORM ABSTRACTION

The framework employs a decoupled, cross-platform architecture bifurcated into Win64 and Nintendo Switch backends. The desktop environment features comprehensive integration of NVIDIA PhysX, ENet networking, and advanced logging, whereas the ARM-based Switch implementation leverages nnSdk, romfs asset routing, and native touchscreen/analog inputs. Subsystem isolation and target selection are strictly enforced at compile time via discrete source-tree compilation paths.

3. WINDOW & RENDERING CONTEXT

The Window abstraction encapsulates the underlying GLFW surface, centralizing viewport clearing, double-buffering routines, and event-polling mechanisms. Efficient asset allocation is achieved through an associative container-based texture cache and specialized OpenGL material loaders. Furthermore, it integrates lifecycle hooks for Dear ImGui, facilitating the seamless rendering of diagnostic overlays and developer tool interfaces.

4. INPUT SYSTEM

The Input system maintains a precise state machine tracking alphanumeric keys, modifiers, and primary mouse inputs across three discrete states: rising-edge triggers (pressed), continuous active states (down), and falling-edge releases (up). Real-time spatial tracking yields precise cursor coordinates. On the console tier, the implementation expands to query analog stick deflection and capacitive multi-touch metrics, including contact validation and coordinates.



5. ENTITY-COMPONENT SYSTEM (ECS)

Architectural modularity is driven by a generic, header-only Entity-Component System (ECS) that facilitates dynamic component registration and monotonic entity identifier allocation. Complex subsystem interactions are queried using variadic templates (`GetEntitiesWithComponents`) to aggregate heterogeneous component archetypes. The structural backbone comprises essential components mapping transformations, rendering, illumination, physics, networking, and kinematics.

6. RENDER PIPELINES

The graphics subsystem supports six distinct runtime-configurable rendering modalities: standard forward, illuminated forward, and forward rendering with omnidirectional shadow maps, alongside a specialized 2D sprite pipeline. The flagship deferred PBR pipeline utilizes a Cook-Torrance BRDF incorporating GGX distribution, Schlick-GGX geometry attenuation, and Fresnel-Schlick approximations under a metallic-roughness workflow.

7. DEFERRED & PBR DEFERRED RENDERING

The deferred rendering pipeline optimizes lighting complexity by decoupling scene geometry from shading equations via a multi-target G-buffer. A geometry pass serializes spatial attributes, normals, and surface metrics, which a subsequent full-screen lighting pass evaluates against dynamic light sources and shadow maps. In the PBR permutation, the G-buffer is adapted to serialize material properties, enabling physically consistent surface evaluation.

8. SHADOW MAPPING

The `ShadowMapSystem` generates high-resolution 1024×1024 depth textures for localized and directional lights, utilizing geometry-shader hardware amplification to render omnidirectional point-light cubemaps in a single pass. To suppress spatial aliasing and jagged silhouettes, the sampling routine combines a 3×3 Percentage-Closer Filtering (PCF) kernel with a stochastic 20-sample Poisson-disk distribution.

9. CAMERA SYSTEM

The Camera subsystem offers a first-person kinematic model governed by yaw and pitch Euler angles, featuring adjustable translational velocity and rotational sensitivity. To optimize rendering efficiency, frustum culling is performed by extracting six clipping



planes from the view-projection matrix to evaluate Axis-Aligned Bounding Box (AABB) intersections. It exposes spatial matrices for transform pipelines.

10. GEOMETRY GENERATION

The Geometries utility provides a static interface for the procedural synthesis of primitive topologies, including cubes, planes, and parameterized UV-spheres. Each generated vertex buffer strictly interleaves coordinate, texture, and normal attributes. The system can emit these structures as either unbuffered primitive streams or as instantiated, material-bound ObjItem containers.

11. SPRITE SYSTEM

The Sprite component instantiates a 2D rendering abstraction that layer-sorts entities equipped with transform and render properties. It handles sprite-sheet animation by calculating dynamic UV offsets derived from parameterized dimensions, columns, and rows. Frame progression is explicitly managed via modular state-mutation controls (setFrame, nextFrame).

12. TERRAIN SYSTEM

The Terrain subsystem procedurally synthesizes vast landscapes via OpenSimplex2 noise layered across five octaves of Fractional Brownian Motion (FBM). Spatial scalability is enforced through a chunk-based Level-of-Detail (LOD) architecture. Surface coloration is driven by a 1D Look-Up Table (LUT) mapping elevation to biomes, while runtime ray-casting queries determine exact world-space heights.

13. MESH & MATERIAL SYSTEM

Every Mesh instance encapsulates discrete OpenGL Vertex Array and Buffer Objects, alongside localized bounding volumes (AABBs) for visibility queries. Surfaces are parameterized by a comprehensive Material structure containing classic Phong parameters, modern PBR metrics, and UV transformation matrices. These components are aggregated hierarchically within an ObjItem container.

14. SHADER & PROGRAM PIPELINE

The shader architecture isolates pipeline compilation through a two-tiered abstraction: the Shader class manages disk I/O and stage compilation, while the Program class handles GPU linking and



uniform-location binding. Supporting a wide array of linear algebra primitives, this interface provides the foundation for systematic shader permutation management.

15. TEXTURE SYSTEM

The Texture subsystem encapsulates asset ingestion and texture generation, utilizing an internal associative cache indexed by file path to prevent redundant disk I/O. By exposing the native GLuint handle, the abstraction allows specialized subsystems to bind assets directly within lower-level rendering stages.

16. PHYSICS SIMULATION (Win64)

The PhysxMaster module encapsulates the NVIDIA PhysX engine within a deterministic, fixed-timestep simulation loop running at 60 Hz. It supports rigid-body primitives in both static and dynamic modalities, automatically reconciling physical transformations back to the ECS registry. Additionally, it offers optimized spatial-query routines for scene ray-casting.

17. NETWORKING (Win64)

Built upon the ENet UDP library, the NetworkManager institutes a robust client-server topology featuring adaptive connection tracking and event-driven peer lifecycle notifications. The system features a dual transmission protocol optimizing for both guaranteed order and low latency. Game state data and gameplay messaging payloads are serialized via byte-aligned network packets.

18. JOB SYSTEM (THREAD POOL)

Task-level parallelism is driven by a JobSystem that scales worker threads to the host platform's hardware concurrency limits. Utilizing a synchronized producer-consumer queue governed by a mutex and condition variable, it executes arbitrary `std::function` payloads. This architecture enables asynchronous asset streaming and asynchronous heavy computing.

19. LOGGING (Win64)

The diagnostic logging layer is exposed via a singleton wrapper around `spdlog`, delivering thread-safe console output demarcated by six severity gradations. It features type-safe, template-variadic string formatting alongside runtime verbosity controls and automated



stream flushing for warnings or critical errors. Emitted diagnostic strings follow a rigorous metadata format.

20. SCRIPTING (Lua via sol2)

High-level behavior and rapid iteration are facilitated through an embedded Lua engine bound via the sol2 library. The binding layer exposes foundational linear algebra primitives, specific ECS component attributes, and structural input queries. This configuration enables runtime manipulation of entity transformations and camera trajectories without recompilation.

21. OBJ LOADER

The asset pipeline relies on tinyobjloader to parse Wavefront geometries from platform-abstracted storage paths, extracting multi-attribute vertex streams and surface properties. Material parsing generates corresponding lighting profiles, while spatial parsing builds mesh primitives. High-volume loading operations are offloaded to the JobSystem to yield asynchronous, thread-safe smart pointers.

22. CARD GAME MODULE

The gameplay logic subsystem incorporates a digital implementation of a Spanish 48-card deck, utilizing a stochastic Fisher-Yates algorithm for uniform distribution. It codifies custom rulesets, containing trick resolution, multi-card dealing mechanisms, and standard Brisca scoring structures. Multiplayer synchronization is maintained by serializing state transitions into distinct network packets.

23. DEBUG SYSTEM

The diagnostic subsystem exposes source-aware macro extensions to catch graphics API faults alongside an integrated GL_ARB_debug_output callback mechanism. This setup registers driver-level events categorized by source, type, and severity. By providing instantaneous tracing of state errors, it dramatically simplifies the identification of low-level graphics pipeline failures.

24. DEPENDENCIES

The architecture minimizes reinventing core primitives by consolidating an industry-standard external ecosystem. Window management, API binding, and mathematical primitives rely on



GLFW, GLAD, and GLM respectively. Core engine features leverage specialized frameworks including Dear ImGui, ENet, NVIDIA PhysX, and sol2, ensuring deterministic behavior across supported runtime configurations.



2.- Problem solving

WINDOW & RENDERING CONTEXT

One of the key concepts in this context is the Swap Chain, which is designed to prevent the user from noticing the rendering process. This is due to the mismatch between the frame rate and the monitor's refresh rate.

To achieve this, the swap chain combines some buffering techniques:

- **Double buffering:** Two video memory regions are used: the *front buffer*, for the display and the visible portion, and the *back buffer*, where the GPU renders the scene.
The way it works is simple: the front buffer is displayed on screen, whilst in the background, the back buffer is rendering the next frame. When the back buffer has finished rendering, they are swapped, and the new front buffer is now displayed.
- **V-Sync:** The aim is to force the GPU to wait for the monitor's vertical blanking signal before swapping buffers. However, if the GPU takes longer than expected to render, it will skip a vertical blanking cycle, causing the screen to stutter.

Next, going into more detail, it is important to understand what the OpenGL rendering context is, as it encapsulates all the resources, shaders and settings of the graphics execution pipeline. Its main functions are state management (as it acts as a state machine, remembering certain settings), multithreading control, and acting as an intermediary between the rendering and the functions.

Finally, it is also important to include development interfaces to facilitate runtime debugging. In the case of our engine, we have used *ImGui*, which operates independently without interfering with the engine's logic.

ImGui's graphics pipeline has two key components:

- **ImGui_ImplOpenGL3_NewFrame:** This call calculates the delta time, updates the mouse's logical coordinates and prepares the internal data structures to begin recording the drawing commands issued by the engine systems.
- **ImGui_ImplOpenGL3_RenderDrawData():** This is the rendering call that takes place just before the SwapBuffers operation. Here, ImGui compiles all the panels, text and buttons into a single large buffer.



INPUT SYSTEM

The input system records data from hardware devices such as keyboards, mice and controllers. The main objective is to convert asynchronous signals into discrete, standardised logical outputs suitable for use in game logic systems such as Lua or ECS.

Generally speaking, we always take into account two data collection approaches that help us resolve issues relating to inputs:

- **Input Polling:** It involves checking the state of a device instantly at some point during the frame's logic, such as with `isKeyDown(W)`.
This method is recommended for its predictability and simplicity at the code level; however, it presents a major problem that occurs when the user makes an input very quickly, with a duration shorter than that of a frame (this can occur during significant drops in FPS in video games), meaning that the input would be ignored completely.
- **Event-Driven Input:** Each interaction generates an event that is added asynchronously to an intermediate message queue. The engine then processes this queue in one go using callbacks.

On the other hand, another fairly common issue when handling inputs can be noise generated by hardware imperfections, such as an internal contact that bounces microscopically.

Therefore, *debouncing* or noise filtering is very important. Generally, this technique is handled internally by the hardware itself or by high-level APIs, but when working at a low level, we need to implement it via software, so that pulses lasting between 5 and 15 milliseconds are discarded.

Another issue may be the repeat rate, so it is helpful to create a delay until the next repeat, followed by a repeat ratio. This is often important when moving characters, for example.

Finally, another concept to take into account in this part of the development is the *dead zone*, which is implemented using a simple numerical range between zero and decimal values very close to zero, to prevent faulty hardware, such as a joystick that automatically tilts to one side, from generating unwanted inputs.



ENTITY-COMPONENT SYSTEM (ECS)

Our implementation of the entity-component system follows the data-oriented programming hierarchy.

To implement an ECS, we can follow two well-known memory layouts: AoS (Array of Structures) and SoA (Structure of Arrays).

- **AoS:** This is the hierarchy in the traditional approach to object-oriented programming, meaning that if a structure contains certain values, such as position, rotation or scale, they are all stored together for a single entity. Although this is a simple and intuitive approach, if a system only wishes to change the position, the CPU is forced to load the rest of the values as well, wasting memory.
- **SoA:** This is more appropriate for data-driven design. The aim is to place each variable in its own separate array.

Modifying, adding or removing components, or destroying entities whilst the system is running, can lead to memory risks and performance issues (due to continuous reallocations). To avoid this problem, *deferred operations* are used: when a modification to an entity or component is requested, the ECS does not alter the memory immediately, but instead places the request in an internal command queue, commonly known as a command buffer or deferred queue.

Finally, one error that can cause major problems is invalidating iterators, since if you iterate, for example, with a `std::vector` and, halfway through the loop, the system deletes or modifies the vector, triggering an internal reallocation, the pointers will point to empty or freed memory, causing a segmentation fault.

RENDER PIPELINES / DEFERRED & PBR DEFERRED RENDERING

To address both of these points, it is crucial to explain the architecture of the rendering pipeline from the perspective of hardware efficiency, memory management and mathematics.

The engine's rendering pipeline is centralised in `Engine::renderScene()` using a switch-case structure based on an



enum class, thereby eliminating the overhead of virtual functions when switching between modes such as Forward, Shaded or Deferred.

Next, the main pipeline implements *deferred shading* to decouple the geometry from the lighting; in an initial geometry pass, the position, normals and albedo are stored in a multi-purpose G-buffer.

On the other hand, in its PBR variant, the G-Buffer is adapted to serialise physical properties of the material, such as Albedo, Metallic and Roughness, by evaluating the Cook-Torrance BRDF during the lighting pass. This calculation incorporates the Trowbridge-Reitz (GGX) microfacet distribution, Schlick-GGX geometric attenuation, the Fresnel-Schlick approximation and the principle of energy conservation, achieving a visually consistent and realistic representation of surfaces.

As for the issues that have come up, the engine initially used forward rendering for all lights, which meant that adding multiple dynamic lights caused performance to drop significantly. This was because, in forward rendering, each object must be rendered once for every light that affects it, executing the full fragment shader for every pixel of every object.

The solution, therefore, is to implement a deferred pipeline with Multiple Render Targets, splitting the rendering process into two phases:

- **Geometry pass:** All the geometry in the scene is rendered once, writing essential variables such as world space position, surface normals, albedo colour, etc., to the G-buffer. We achieve this by using `glDrawBuffers` to write simultaneously to multiple colour attachments.
- **Lighting pass:** Render a quad that displays the textures from the G-buffer and evaluates the lighting equation for each visible pixel. This results in a fixed cost, regardless of the complexity of the geometry.

Finally, by removing redundant shading on non-visible geometry, performance has improved significantly when using a larger number of lights, furthermore, the PBR variant allows roughness and metallic maps to be included for physical evaluation without the need to re-run the vertex shaders.



At the same time, issues have also arisen with shadow mapping, and solutions have been implemented. Thanks to an implementation technique using a geometry shader, it has been possible to generate the complete depth cubemap in a single draw call.

The geometry shader receives each geometry primitive and duplicates it six times, assigning each copy to a different render layer via the `gl_Layer` variable.

SHADOW MAPPING

Implementing dynamic shadows in the engine presented a number of challenges.

The first one was the need to support three different types of lights, each with distinct projection requirements. Consequently, each would require a perspective projection based on the angle of the light cone. However, point lights emit light in all directions, so they require a six-sided depth cubemap.

The problem comes up when you first try to use a single shadow map pipeline for all of them, resulting in artefacts and poor performance.

However, the solution is to create two separate pipelines: one using a 2D shadow map for directional and spot lights, and another using a cubemap with a geometry shader for point lights.

Aliasing issues also arose when rendering shadow maps at a resolution of 1024x1024 and filtering `GL_NEAREST`, causing the edges to appear very jagged.

The solution was to implement manual Percentage-Closer Filtering in the shaders, as `GL_TEXTURE_COMPARE_MODE` had not been enabled to maintain control over the sampling.

For 2D shadow maps, 9 samples have been used around the texture coordinates, spanning a range from -1 to +1 texel in X and Y, comparing each sample with the current depth, and averaging the results. For cubemaps, the situation is more complex because we do not have 2D UV coordinates, but rather a 3D vector. 20 sampling directions have been implemented, uniformly distributed in space, creating a sphere of directions.

Finally, the sampling radius is dynamically scaled according to the distance from the camera: $\text{diskRadius} = (1.0 + \text{viewDistance} / \text{farPlane}) / 25.0$, ensuring that distant objects use a larger radius (resulting in more blur) whilst nearby objects remain sharp, thereby reducing banding in distant shadows without sacrificing detail in nearby ones.



GEOMETRY GENERATION

When this feature was first introduced, it was quite costly and time-consuming to generate a simple cube geometry because, initially, only a vertex vector was available. This meant that a mesh had to be created manually, a material assigned to it, its VAO attributes configured, and then it had to be packaged into an `ObjItem` in order to assign it to a `RenderComponent`.

However, the solution was to receive a reference to a `Window`, a `firstTime` flag, and a texture ID, and to return a complete `ObjItem` with its mesh already created and two preconfigured materials. This reduced the amount of code required to generate the geometry to just a single line. (The resulting `ObjItem` is assigned directly to the render component without the need to configure VAOs or VBOs manually.)

On the other hand, another error we encountered was the repetitive and redundant call to `glfwMakeContextCurrent`, which was being called every time a mesh was loaded.

In this case, the solution was to implement a boolean flag set to false when the first mesh was constructed, as this would be the one establishing the GL context. All subsequent meshes would then skip the call to `glfwMakeContextCurrent`.

MESH & MATERIAL SYSTEM

When an object is moved, for example, when returning it from the OBJ loading function or when generating basic geometry, the `unique_ptrs` vector needs to be reallocated. This would require the compiler to copy the pointers, which is not permitted in this case. Furthermore, OpenGL VAO and VBO buffers are resources that cannot be duplicated; therefore, if a mesh is copied, there would be two objects that would need to delete the same VAO in their destructors, causing potential crashes.

The solution to this problem is simple: a move constructor and a move assignment were implemented in `Mesh` to transfer ownership of the VAO and VBO from the source to the destination, and to set the source's VAO and VBO to zero so that their destructors do not free them.

Furthermore, a very basic error that occurred at one stage during development due to a lack of defensive programming was the failure to initialise the material fields with a safe value, resulting in the generation of garbage and even causing crashes if that value was binded as a texture.



Finally, the solution is simply to initialise the fields with safe values in the default constructor, ensuring that any unconfigured material has a predictable neutral appearance and does not corrupt the render pipeline.

SHADER & PROGRAM PIPELINE

When working with OpenGL in this context, it is worth noting that this API requires a compilation process consisting of two separate stages. First, each shader stage (vertex, geometry and fragment) is compiled separately using *glCompileShader*, and then they are linked together into a program object using *glLinkProgram*.

The problem comes up when you try to combine these stages, for example, attempting to link an uncompiled shader or access uniforms before linking, which results in errors that are difficult to debug. Initially, the engine had a single class that combined both stages, which caused linking errors.

However, the solution was to split the functionality into two classes with clearly defined responsibilities: the first of these, “Shader”, is responsible for reading from disk, compiling GLSL and checking for compilation errors.

The second class, called “Program”, takes one or more pre-compiled shader objects, attaches them using *glAttachShader*, links them using *glLinkProgram*, and exposes the attributes and uniforms for configuration at runtime.

Finally, this fix ensures that the OpenGL pipeline runs in the correct order and that any errors that arise are detected at the appropriate stage.

LOGGING (Win64)

When we started developing the logger, we realised that if we wanted it to cover all types of data, we would need many identical functions for each type, which would result in extremely long and repetitive code.

However, the logger is built on top of the *spdlog* library, which also integrates the *fmtlib* formatting engine, allowing strings to be interpolated at compile time using placeholders, rather than traditional *printf* format specifiers such as *%d*, *%s* or *%f*.

In short, what this achieves is a single function that allows you to print any type of variable using a placeholder.



To go into a little more detail, here's what happens: at compile time, the format string is parsed and the number of placeholders is counted. It then checks that the number of placeholders matches the number of arguments, before confirming that each argument type is valid according to our specification. Finally, the logger sends the error so that the user can see it.

Furthermore, although it wasn't strictly speaking a bug, the function that initialises the logger was also changed so that, rather than having to initialise it in every file where it is used, it is initialised just once in the Engine, which is good practice for keeping the code clean

CARD GAME MODULE

Implementing the card game has required integrating the logic behind most of the engine's features, which has presented many challenges and problems.

Probably the most significant challenge has been the multiplayer logic, as it raises many questions about how everything should interact with one another.

Particular attention must be given to execution order issues, as in multiplayer games, packets and payloads must be sent and received in the correct order, consequently, if an error of this kind occurs, a client may attempt to receive a packet that is not yet ready to be sent, even if the error involves just a single frame. For this reason, we have tried to maintain a specific, well-structured order throughout the demonstrator, so that all the information can be managed correctly.

Furthermore, it is also important to take into account the client-server architecture, ensuring that we always know which entity is responsible for each task.

In our case, the method of dealing cards in the multiplayer game posed a major challenge when it came to planning and implementing it in code, however, the final solution was to manage this logic via the server, as if it were handled by each client, everyone would have the same cards since the seed would be identical for all.

Finally, one of the last issues was ensuring that all players could see one another. This was because, initially, players were only able to see clients who had already logged in before them, due to a bug that



failed to update the data packets containing information about the other meshes for users who were already connected when they entered the game.



3.- Rendering techniques documentation

Below I provide a tutorial-style explanation of every rendering technique I have implemented in the engine. For each technique, I describe how it works, what its underlying principles are, and how I integrated it into the codebase. This document demonstrates that I understand what I have implemented and can explain it clearly.

1. FORWARD RENDERING (TEXTURED)

Forward rendering is the simplest rendering approach I implemented. For each object in the scene, the vertex shader transforms its vertices from local space to clip space using the model and view-projection matrices ($gl_Position = (VP * model) * vec4(position, 1.0)$). The fragment shader then samples a diffuse texture at the interpolated UV coordinates and outputs the resulting colour. I also apply UV transformations: "uvOffset" and "uvScale" uniforms allow materials to scroll or tile their textures ($vec2 transformedUV = fragTexCoords * uvScale + uvOffset$). Fragments with an alpha value below 0.1 are discarded to implement basic transparency. This pipeline is defined in "textured_obj_vertex.txt" and "textured_obj_fragment.txt", and the C++ side iterates over all entities with "RenderComponent" and "TransformComponent" to draw them one by one.

2. FORWARD LIGHTING (PHONG REFLECTION MODEL)

I extended the basic forward renderer with per-fragment Phong lighting. The vertex shader computes "fragWorldPosition" by transforming vertices with the model matrix and "fragNormal" using the inverse-transpose of the model matrix ($mat3(transpose(inverse(model))) * normal$) to preserve normal orientation under non-uniform scaling. In the fragment shader, I classify lights by "lightType": type 1 is directional, type 2 is spot, and type 3 is point. For each light, the ambient term ($ambientColor *$



ambientIntensity") provides base illumination. The diffuse term follows Lambert's law: $\text{"max(dot(normal, lightDir), 0.0)"}$, multiplied by the light colour and intensity. The specular term uses the Blinn-Phong reflection model: $\text{"pow(max(dot(viewDir, reflectDir), 0.0), shininess)"}$, where $\text{"reflectDir = reflect(-lightDir, norm)"}$. The final colour is $\text{"texColor * (ambient + diffuse + specular)"}$. All three light types are accumulated additively using "GL_ONE, GL_ONE" blending, with the ambient pass rendered first and subsequent passes using "GL_EQUAL" depth testing to layer light contributions correctly.

3. DEFERRED SHADING

Deferred shading decouples geometry processing from lighting calculations. I implemented this in `"RenderDeferredSystem"` using two distinct shader programs. In the first pass (Geometry Pass), I render all opaque geometry into a G-Buffer (see technique 8) that stores per-pixel world-space position, normal, and albedo with specular strength. The second pass (Lighting Pass) renders a fullscreen quad and samples the G-Buffer textures to compute lighting for every pixel. This approach means the lighting calculations run only once per pixel rather than once per pixel per light, making it far more efficient for scenes with many lights. The C++ implementation binds the G-Buffer FBO for the geometry pass, then switches to the default framebuffer for the lighting pass. Each light contributes additively with "GL_ONE, GL_ONE" blending, and I start with an ambient pass before iterating over directional, spot, and point lights.

4. PHYSICALLY BASED RENDERING (COOK-TORRANCE BRDF)

For PBR, I implemented the Cook-Torrance microfacet BRDF model in `"pbr_lighting_fragment.txt"`. The BRDF consists of three components, all defined as GLSL functions:



- "DistributionGGX": the Normal Distribution Function using the Trowbridge-Reitz GGX formulation: $a^2 / (\pi * (N \cdot H^2 * (a^2 - 1) + 1)^2)$, where "a = roughness²". This controls the statistical orientation of microfacets relative to the half-vector H.

- "GeometrySchlickGGX" and "GeometrySmith": the geometry function uses Smith's method, which combines two Schlick-GGX sub-functions — one for the view direction and one for the light direction: $G = G_1(N \cdot V) * G_1(N \cdot L)$. This accounts for microfacet self-shadowing at grazing angles.

- "fresnelSchlick": the Fresnel approximation computes the fraction of light that reflects specularly: $F_0 + (1 - F_0) * (1 - \cos\theta)^5$. The base reflectance "F₀" is set to "vec3(0.04)" for non-metals and is mixed with the albedo for metals ("F₀ = mix(F₀, albedo, metallic)").

The final specular contribution is $(NDF * G * F) / (4 * N \cdot V * N \cdot L + 0.0001)$. The diffuse term is $kD * albedo / \pi$, where $kD = (1 - F) * (1 - metallic)$ — this enforces energy conservation by ensuring that metals reflect most light specularly and absorb very little diffuse energy. The complete PBR equation is $(kD * albedo / \pi + specular) * radiance * N \cdot L$.

5. SHADOW MAPPING (DIRECTIONAL AND SPOT LIGHTS)

Shadow mapping is a two-pass technique. In the first pass, I render the scene from the light's perspective using a light-space matrix ("lightSpaceMatrix") and store the depth values in a 2D texture ("GL_DEPTH_COMPONENT16" at 1024×1024 resolution). The vertex shader "shadow_map_vertex.txt" transforms vertices by "lightSpaceMatrix * model". The fragment shader is empty because OpenGL writes the depth automatically.



In the second pass, during lighting, I transform each fragment's world position into light space: `fragPosLightSpace = lightSpaceMatrix * vec4(fragWorldPosition, 1.0)`, then normalise to `[0,1]` with `projCoords = projCoords * 0.5 + 0.5`. I compare `currentDepth` against the closest depth stored in the shadow map. A shadow bias (`max(0.005 * (1.0 - dot(normal, lightDir)), 0.0005)`) prevents shadow acne by shifting the comparison depth slightly towards the light. The light-space coordinates are clamped to `[0,1]` before sampling, and fragments outside the light frustum (`projCoords.z > 1.0`) are considered unshadowed. Texture border colour is set to white `(1,1,1,1)` so that areas outside the shadow map receive full lighting.

6. OMNI-DIRECTIONAL SHADOW MAPPING (POINT LIGHTS)

For point lights, shadows must be computed in all directions, so I use cube-map shadow mapping. The resource creation is handled by `CreateOmniShadowResource`, which generates a `GL_TEXTURE_CUBE_MAP` with `GL_DEPTH_COMPONENT16` format. The key innovation is the geometry shader (`omni_shadow_geom.txt`), which takes incoming triangles and emits them 6 times — once for each cube face — setting `gl_Layer` to select the face and using one of 6 light-space matrices (`shadowMatrices[6]`). The fragment shader (`omni_shadow_frag.txt`) computes the distance from the fragment to the light position and normalises it by `far_plane`: `gl_FragDepth = distance(fragPos, lightPos) / far_plane`.

In the lighting pass, I sample the cube map using the vector from the fragment to the light (`fragToLight`). The stored depth is multiplied back by `far_plane` for comparison against the actual distance. For point-light PCF, I use 20 sample directions arranged in a fixed pattern and sample the cube map at offsets scaled by `diskRadius`, which increases with distance to simulate softer shadows further away.



7. PERCENTAGE CLOSER FILTERING (PCF)

To produce soft shadow edges instead of hard aliased ones, I implemented Percentage Closer Filtering with a 3×3 kernel. Rather than a single depth comparison, I sample the shadow map 9 times in a 3×3 neighbourhood around the fragment's projected coordinates:

```
""""
```

```
for(int x = -1; x <= 1; ++x)
```

```
    for(int y = -1; y <= 1; ++y) {
```

```
        float pcfDepth = texture(shadowTexture, projCoords.xy +  
vec2(x,y) * texelSize).r;
```

```
        shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
```

```
    }
```

```
shadow /= 9.0;
```

```
""""
```

The "texelSize" is "1.0 / textureSize(shadowTexture, 0)" for the forward renderer, and hardcoded as "1.0 / vec2(1024.0, 1024.0)" in the deferred and PBR renderers since the shadow map resolution is fixed. The result is a smooth shadow factor between 0 and 1 that I multiply into the lighting. This introduces a performance cost of 9 texture lookups per shadowed light, but dramatically improves visual quality.

8. G-BUFFER CONSTRUCTION (DEFERRED SHADING)

I built the deferred shading G-Buffer with three colour attachments and one depth attachment:



- "gPosition" (GL_RGBA16F, location 0): stores world-space position. I use 16-bit floating point for sufficient precision across the scene, and the fourth component remains unused.
- "gNormal" (GL_RGBA16F, location 1): stores world-space normals. 16-bit float ensures accurate specular highlights at grazing angles.
- "gAlbedoSpec" (GL_RGBA8, location 2): stores the albedo colour (RGB) and the average specular strength (A). 8-bit resolution is adequate for colour data.
- A depth renderbuffer (GL_DEPTH_COMPONENT24) attached to "GL_DEPTH_ATTACHMENT".

I use "glCreateFramebuffers" and "glCreateTextures" (DSA) for cleaner resource management. The textures are set to "GL_NEAREST" filtering and "GL_CLAMP_TO_EDGE" wrapping. The G-Buffer is initialised in "InitGBuffer()" and regenerated on window resize via "Resize()". The geometry pass binds this FBO, clears it, and renders all geometry into the textures. The lighting pass then reads from these textures to compute the final image.

9. G-BUFFER CONSTRUCTION (PBR)

The PBR G-Buffer extends the deferred shading G-Buffer with an additional attachment to accommodate the PBR material parameters:

- "gPosition" (GL_RGBA16F, location 0): world-space position.
- "gNormal" (GL_RGBA16F, location 1): world-space normals.



- "gAlbedoMetallic" (GL_RGBA8, location 2): albedo (RGB) and metallic factor (A). Storing metallic in the alpha channel avoids an extra attachment.

- "gRoughness" (GL_R8, location 3): a single-channel 8-bit texture for the roughness value. I chose "GL_R8" because roughness is a scalar between 0 and 1 and does not require more precision.

The PBR lighting pass reads all four textures and feeds them into the "CalculatePBR()" function: position determines the view and light vectors, normal provides the surface orientation, albedo provides the base colour, metallic modulates the Fresnel " F_0 ", and roughness drives both the GGX distribution and the Smith geometry function.

10. LAMBERTIAN DIFFUSE MODEL

The Lambertian diffuse model is the foundation of diffuse lighting in both my Phong and PBR implementations. It states that the reflected intensity is proportional to the cosine of the angle between the surface normal and the incoming light direction: " $\text{diff} = \max(\text{dot}(\text{normal}, \text{lightDir}), 0.0)$ ". The "max" function clamps negative values to zero, ensuring that surfaces facing away from the light receive no diffuse contribution. In my forward lighting shader, this is multiplied by " $\text{lightColor} * \text{lightIntensity}$ ". In the PBR shader, it is part of the full BRDF: the diffuse lobe is " $kD * \text{albedo} / \pi * N \cdot L$ ", where the division by π ensures energy conservation (the Lambertian BRDF integrates to 1 over the hemisphere). The Lambertian model assumes perfectly diffuse (matte) surfaces that scatter light equally in all directions regardless of viewing angle.



11. SPOT LIGHT CONE ATTENUATION

I implemented spot lights with a smooth cone-shaped falloff. For a spot light, I compute the angle "theta" between the light-to-fragment direction and the spot's forward direction: "theta = dot(lightDir, normalize(-spotLightDir))". The light has two cutoff angles: "spotCutOff" (inner cone, full intensity) and "spotOuterCutOff" (outer cone, zero intensity). I compute an intensity factor using smooth interpolation:

```
""
```

```
float epsilon = spotCutOff - spotOuterCutOff;
```

```
float intensity = clamp((theta - spotOuterCutOff) / epsilon, 0.0, 1.0);
```

```
""
```

This creates a soft transition from fully lit to completely dark, avoiding the harsh edge that a single cutoff would produce. The final light contribution is multiplied by this intensity value. In both the forward and deferred/PBR renderers, the spot-specific uniforms ("spotLightDir", "spotCutOff", "spotOuterCutOff") are set via "glUniform3f" and "glUniform1f".

12. LIGHT ATTENUATION (CONSTANT, LINEAR, QUADRATIC)

I implemented physically-based light attenuation for point and spot lights using the quadratic inverse-distance formula:

```
""
```

```
float distance = length(lightDirOrPos - fragWorldPosition);
```

```
float attenuation = 1.0 / (constant + linear * distance + quadratic * distance * distance);
```



.....

The three coefficients serve distinct purposes:

- "constant" (usually 1.0): prevents division by zero and controls the base intensity at the light source.
- "linear": controls how quickly the light fades with distance (linear falloff).
- "quadratic": models the physical inverse-square law of light propagation.

I reuse the same uniform names ("spotConstant", "spotLinear", "spotQuadratic") for both spot and point lights in all three renderers. The attenuation factor is multiplied into the diffuse and specular contributions before blending.

13. AMBIENT LIGHTING

Ambient lighting provides a base level of illumination that approximates indirect light bouncing around the scene. I implemented it as a uniform term: "ambientColor * ambientIntensity", multiplied by the albedo (in the forward and deferred renderers) or by the albedo directly (in the PBR renderer, where the ambient pass simply adds "ambientColor * ambientIntensity * albedo" to the result).

In my system, ambient lighting is optional and controlled by the "useAmbient" uniform. The "LightComponent" stores an "ambient_" struct with a colour and intensity, and the engine UI allows the user to configure it. In the forward renderer, the ambient pass is rendered first with "glDepthMask(GL_TRUE)" and additive blending disabled, then subsequent light passes use "glDepthMask(GL_FALSE)" and "GL_EQUAL" depth testing. In the deferred and PBR renderers, the



ambient pass is simply the first fullscreen quad rendered with "lightType = 0".

14. PROCEDURAL TERRAIN GENERATION

I implemented procedural terrain generation using the "FastNoiseLite" library with OpenSimplex2 noise and FBM (Fractional Brownian Motion). The terrain is generated in chunks, each chunk being a grid of vertices whose heights are determined by sampling the noise function.

The FBM implementation uses 5 octaves with a lacunarity of 2.0 and a gain of 0.5:

- Octave 0: base noise at full amplitude.
- Octave 1: noise at 2× frequency, 0.5× amplitude.
- Octave 2: noise at 4× frequency, 0.25× amplitude.
- ...and so on.

This produces natural-looking terrain with both large-scale features (hills, valleys) and fine detail (rocks, bumps). The height values are scaled by "maxHeight" and stored in the vertex positions. Normals are computed from the triangle geometry using cross products of edge vectors.

The terrain is stored as an "ObjItem" with multiple named meshes following the pattern ""terrain_%d_%d_nc%d_lod%d_wx%f_wz%f"", where the parameters encode the grid position, chunk count, LOD level, and world offset.



15. TERRAIN LEVEL-OF-DETAIL (LOD)

To maintain performance at large viewing distances, I implemented a chunk-based LOD system with four discrete levels: "[1, 2, 4, 8]" — these values represent the vertex step size (1 = full detail, 8 = lowest detail).

The LOD selection logic computes the camera's grid position relative to the terrain chunks and then determines the ring distance:

```
""""  
  
int ringDist = max(abs(gx - camGridX), abs(gz - camGridZ));  
  
if (ringDist <= 7) desiredLod = 0;  
else if (ringDist <= 12) desiredLod = 1;  
else if (ringDist <= 17) desiredLod = 2;  
else desiredLod = 3;  
  
""""
```

Only the mesh with the matching LOD level for the camera's ring distance is rendered. The four LOD meshes share the same height data but use fewer vertices at lower levels, reducing GPU memory bandwidth and vertex processing costs for distant terrain. The C++ parsing is done with "sscanf_s", extracting grid coordinates and LOD level from the mesh name.

16. HEIGHT-BASED COLOUR LOOK-UP TABLE (LUT)

I implemented a 1D texture LUT to colour the terrain based on elevation. The LUT is a 256×1 texture where each texel represents a



colour for a given normalised height. The terrain fragment shader samples it conditionally when "useHeightLUT" is true:

```
""
```

```
float h = clamp(fragPosition.y / maxHeight, 0.0, 1.0);
```

```
texColor = texture(diffuseTexture, vec2(h, 0.5)).rgb;
```

```
""
```

The "diffuseTexture" uniform is reused as the LUT sampler — the material system sets either a regular texture or the LUT texture. The "maxHeight" uniform normalises the terrain height to "[0, 1]". This allows mapping low elevations to brown/green (earth/grass), mid elevations to grey (rock), and high elevations to white (snow), creating a visually appealing terrain colour gradient without requiring a separate colour map texture per vertex.

17. FRUSTUM CULLING

I implemented view-frustum culling in "RenderLightsSystem::DrawCall" to avoid rendering geometry outside the camera's view. The frustum planes are extracted from the view-projection matrix "vp". Each plane is defined by a normal and a distance "d", extracted using the standard Gribb-Hartmann method:

```
""
```

```
frustum.planes[0].normal = vec3(vp[0][3] + vp[0][0], vp[1][3] +  
vp[1][0], vp[2][3] + vp[2][0]);
```

```
frustum.planes[0].d = vp[3][3] + vp[3][0];
```

```
// ... for all 6 planes (left, right, top, bottom, near, far)
```



```
""""
```

Each plane's normal is normalised after extraction. For each mesh, I compute an AABB in world space by scaling the mesh's local AABB by the entity's scale and offsetting by its position. I then check "IsAABBInFrustum(frustum, worldAabbMin, worldAabbMax)" — if the AABB is entirely outside any plane, the mesh is skipped. This reduces draw calls significantly in scenes with many objects spread across a large area.

18. GEOMETRY SHADER (CUBE-MAP RENDERING)

For omni-directional shadow mapping, I implemented a geometry shader ("omni_shadow_geom.txt") that renders the scene to all six cube-map faces in a single pass. The shader takes triangles as input and emits a triangle strip with up to 18 vertices (6 faces × 3 vertices):

```
""""
```

```
for(int face = 0; face < 6; ++face) {  
    gl_Layer = face;  
    for(int i = 0; i < 3; ++i) {  
        FragPos = gl_in[i].gl_Position;  
        gl_Position = shadowMatrices[face] * FragPos;  
        EmitVertex();  
    }  
    EndPrimitive();  
}  
""""
```



The "gl_Layer" built-in variable selects which cube-map face to render to (0 through 5 for positive X, negative X, positive Y, negative Y, positive Z, negative Z). The six "shadowMatrices" are precomputed in C++ using "glm::lookAt" from the light position towards each axis direction, combined with a 90-degree perspective projection. This approach avoids six separate draw calls, reducing CPU-GPU communication overhead.

19. SPRITE AND 2D RENDERING

I implemented a 2D sprite system for rendering billboarded or screen-space elements. The "Sprite" class stores an entity ID ("ecsEntityIdentifier_"), a layer value ("layer_") for draw ordering, and holds a "RenderComponent" pointer. Sprites are managed through the "MotardaEng" class using "generateSprite".

The sprite system uses orthographic projection for 2D rendering. Sprites can be rendered either in a separate 2D pass (using "RenderType::Bidimensional") or combined with 3D elements. The sprite mesh is a simple quad with UV coordinates, textured with the sprite's texture atlas. The layer value determines the rendering order — sprites with higher layer values are rendered on top. The engine's "renderScene()" function handles 2D sprites separately from the 3D scene when "Bidimensional" mode is active.

20. SPRITE SHEET ANIMATION

I extended the sprite system with sprite-sheet animation support through "generateSpriteSheet". The animation system tracks:

- "frameWidth_" and "frameHeight_": the size of each individual frame in pixels.



- "columns_" and "rows_": the layout of frames in the texture atlas.
- "totalFrames_": total number of frames ("columns × rows").
- "currentFrame_": the currently displayed frame index.

The "setFrame(int frame)" method computes the UV coordinates for a given frame index by dividing the frame position by the total columns and rows. The "nextFrame()" method advances to the next frame, wrapping around when the last frame is reached. UV manipulation is handled at the mesh or shader level to select the correct region of the texture atlas, effectively animating the sprite without changing the bound texture. This is efficient for 2D character animation and particle effects.



4.- Task distribution

Throughout the year, we have tried to divide the work fairly and equally, whilst also taking into account the level of knowledge and preferences of both students.

With regard to the above, David Rial had a particular interest in rendering techniques, and although both of us had worked on this, he was more keen on it, so we decided that, to streamline the work and save time, it would be a good idea for him to focus on those tasks. However, Iván Galán wanted to expand his knowledge and delve deeper into networking implementation, which would prove very useful for subsequently creating some of the demonstrators more efficiently.

Speaking of the final demos, two have been created that incorporate virtually all the features added to the engine. The first is a multiplayer card game simulating Brisca, a Spanish card game, created by Iván Galán, whilst the procedural terrain demo with free movement was created by David Rial.

Other aspects worth highlighting regarding the development include, for example, David's significant contribution to engine restructuring thanks to his high work rate, whilst in the background, his teammate implemented other features into the engine

However, Iván's commitment to keeping users informed has enabled him to take charge of the documentation and aspects such as the logger, which improves the "quality of life" of working in the engine.

In conclusion, although we each had our own specialties and strengths, we both tried to contribute to most of the features to a greater or lesser extent, whilst also understanding each other's code.



5.- Bibliography

OpenGL Context. Online. Khronos Group Wiki. [2025-02-10]. Available from: https://www.khronos.org/opengl/wiki/OpenGL_Context. [viewed 2026-05-23].

Deferred Shading. Online. LearnOpenGL. [n.d.]. Available from: <https://learnopengl.com/Advanced-Lighting/Deferred-Shading>. [viewed 2026-05-23].

Sol2 Documentation: Tutorial. Online. The Sol2 Contributors. [2025-08-22]. Available from: <https://sol2.readthedocs.io/en/latest/tutorial/all-the-things.html>. [viewed 2026-05-24].

NIX, Spencer. OpenSimplex2. Online. GitHub. 2024. Available from: <https://github.com/KdotJPG/OpenSimplex2>. [viewed 2026-05-24].

DE VRIES, Joey. Physically Based Rendering: Theory. Online. LearnOpenGL. [n.d.]. Available from: <https://learnopengl.com/PBR/Theory>. [viewed 2026-05-26].

GGX Shader Creation. Online. Stack Overflow. 2018. Available from: <https://stackoverflow.com/questions/53825287/ggx-shader-creation>. [viewed 2026-05-26].